Project 1: Pact Contract Testing Report

Objective

Validate the contract between the **POS Consumer** and the **OMS Provider** using Pact. The implementation verifies both a successful response (200 OK) and a negative scenario (404 Not Found) to ensure the consumer and provider remain compatible.

Components

1. Consumer (OmsClient.java)

- get200(int id) – Retrieves an existing order and validates a 200 OK response.
- get404(int id) – Requests a missing order and validates a 404 Not Found response.

---

2. Consumer Pact Test (PosOmsConsumerPactTest.java)

Scenario 1: Order Exists

Provider State

Order 123 exists

Request

GET /order/123

Expected Response

Status : 200
Content-Type : application/json

```
{
  "orderId": 123,
  "status": "CONFIRMED",
  "total": 42.0
}
```

Assertions

- Status Code = 200
- Order Id = 123
- Status = CONFIRMED

- Total = 42.0

Scenario 2: Order Not Found

Provider State

Order 203 not exists

Request

GET /order/203

Expected Response

Status : 404
Content-Type : application/json

```
{
  "status": "Product not found"
}
```

Assertions

- Status Code = 404
- Response Body

status = Product not found

Provider Verification (OmsProviderVerification.java)

Purpose

Verifies that the OMS Provider satisfies the contract published by the consumer.

Provider State 1

Order 123 exists

WireMock Stub

GET /order/123

Response

```
{
  "orderId":123,
  "status":"CONFIRMED",
```

"total":42.0
}

Status

200 OK

Provider State 2

Order 203 not exists

WireMock Stub

GET /order/203

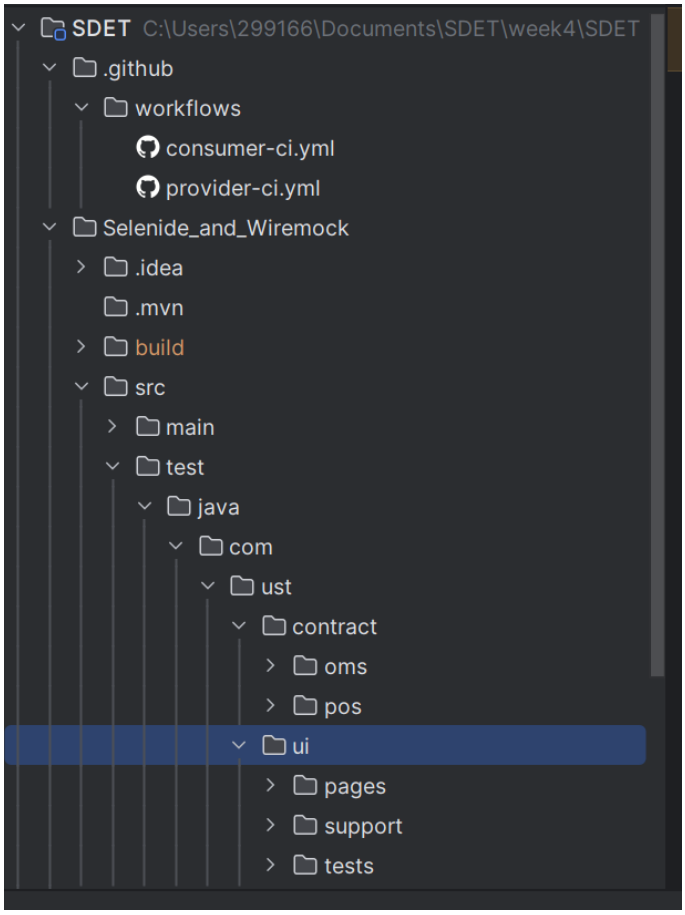
Response

{
"status":"Product not found"
}

Status

404 Not Found

Project Structure



Test Scenarios Covered

Scenario	Request	Expected Status	Result
Existing Order	GET /order/123	200	Pass
Missing Order	GET /order/203	404	Pass

Technologies Used

- Java 22
- JUnit 5
- Pact JVM V4
- PactFlow
- WireMock
- RestAssured
- Maven

Outcome

The implementation successfully validates both positive and negative contract scenarios.

Positive Contract

- Existing order returns **200 OK** with the expected JSON payload.
- Consumer deserializes the response correctly.

Negative Contract

- Missing order returns **404 Not Found**.
- Consumer verifies both the HTTP status code and the error message.

Provider Verification

- WireMock simulates the OMS Provider.
- Provider states (Order 123 exists and Order 203 not exists) satisfy the consumer contract.
- Pact ensures both consumer expectations and provider implementations remain synchronized, preventing breaking API changes.

Conclusion

This contract testing implementation demonstrates how Pact can be used to verify API compatibility between services. By covering both successful (200 OK) and failure (404 Not Found) interactions, it improves confidence in service integration, enables early detection of breaking changes, and supports reliable CI/CD pipelines.

Project 2: Selenide UI Testing

Objective

To automate the product catalog workflow using Selenide and validate that users can search for products, navigate to the product details page, and verify product availability.

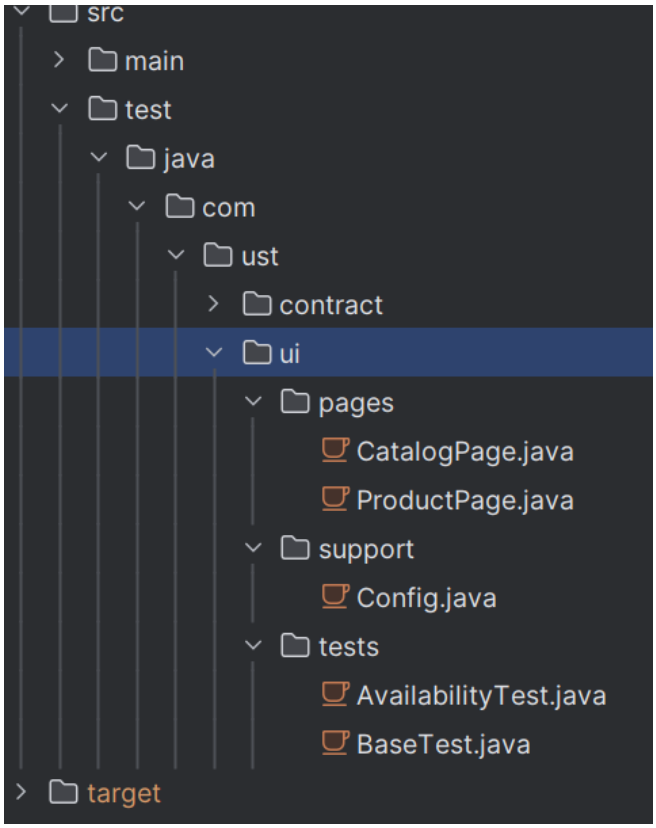
User Story

As a customer, I want to search for a product from the catalog and view the details so that I can verify the stock availability before purchasing.

Overview

This project implements UI automation using the Selenide framework and the Page Object Model (POM) design pattern. The test opens the product catalog, searches for a product, navigates to the product details page, and verifies that the product stock is available for purchase.

Project Structure:



Test Scenario

Scenario: Verify Product Availability

Steps:

1. Open the Product Catalog page.
2. Verify that the catalog page is displayed.
3. Search for a product using the keyword "sho".
4. Verify that matching products are displayed.
5. Open the first product from the search results.
6. Verify the Product Details page.
7. Verify that the Add to Cart button is enabled.
8. Verify that the availability status is "In stock".

Results

- Product catalog loaded successfully.
- Product search completed successfully.
- Product details page opened successfully.
- Product availability verified successfully.

Conclusion:

This project successfully automated the product search and verification workflow using the Selenide framework. The test validated the catalog page, product search, navigation to the product details page, and product availability. Using the Page Object Model and Selenide's built-in waits resulted in a simple, maintainable, and reliable UI automation solution.